Lambdas

Capturing vs fat-free (non-capturing)

06

What is a lambda?

An anonymous function that implements a functional interface such as `Predicate`.

- No `$` class file: `javac` emits an `invokedynamic` call.
- The runtime (`LambdaMetafactory`) builds the class on first use.
- Whether it allocates depends on what it captures.

Capturing (stateful) lambdas

Uses a local, a parameter or `this` from the enclosing scope.

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return list.stream().anyMatch(each -> each.equals(target));  // new object per call
}
```

- Each evaluation creates a new object that holds the captured values.
- Captured locals must be effectively final.

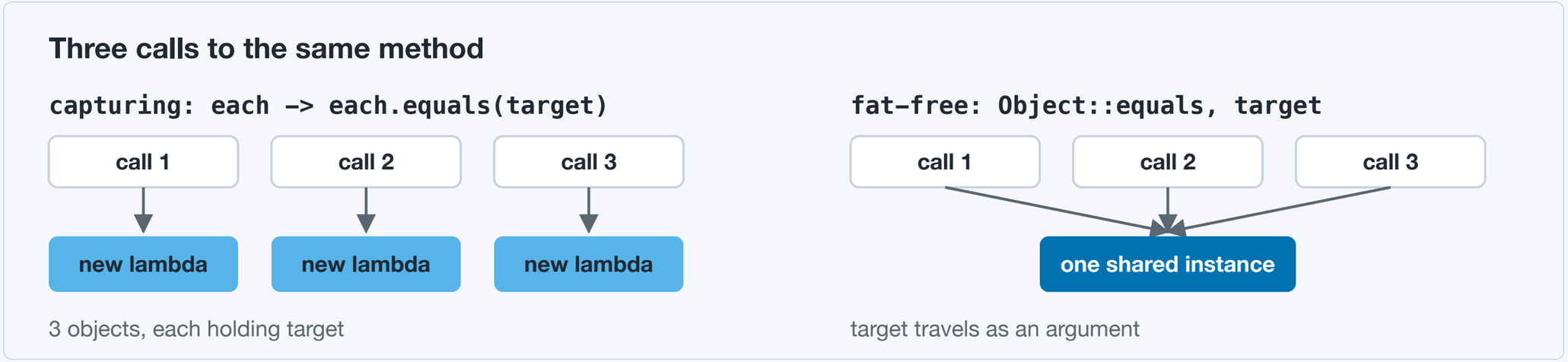
Fat-free (non-capturing) lambdas

Uses only its own parameters, constants and static members.

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return Iterate.anySatisfyWith(list, Object::equals, target);  // one shared instance
}
```

- One instance is created and then reused on every call.
- Extra values travel as arguments (the `*With` methods).

Capturing vs fat-free



- With escape analysis off, only the arguments remain (here the `Integer` box).

References

- The Java Tutorials: Lambda Expressions
- JLS SE 25, 15.27: Lambda Expressions (15.27.2 effectively final, 15.27.4 evaluation)
- `LambdaMetafactory` (Java SE 25 API)
- Java 8: Lambdas, Part 1 (Oracle technical article)